DATA STRUCTURES

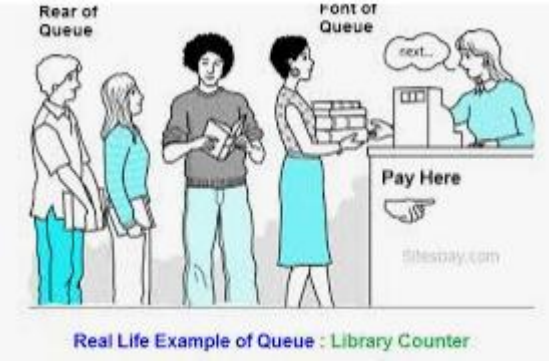
UNIT-2

Dr. SELVA KUMAR S
PROFESSOR & Associate Head
B.M.S. COLLEGE OF ENGINEERING



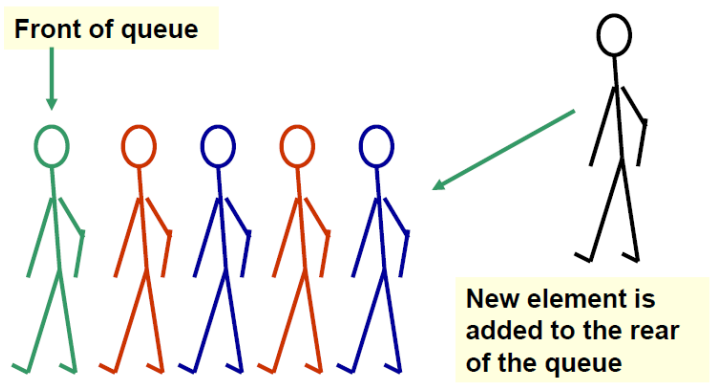
QUEUE -ADT

- **Queue: A list or a collection with the restrictions that insertion can be performed at one end (the *rear* or *tail* of the queue) and deletion can be performed at other end (the *front* or *head* of the queue).**
- A queue is a ***FIFO*(first in, first out) data structure**
  - Any waiting line is a queue:
  - The check-out line at a grocery store
  - The cars at a stop light
  - Ticket counter

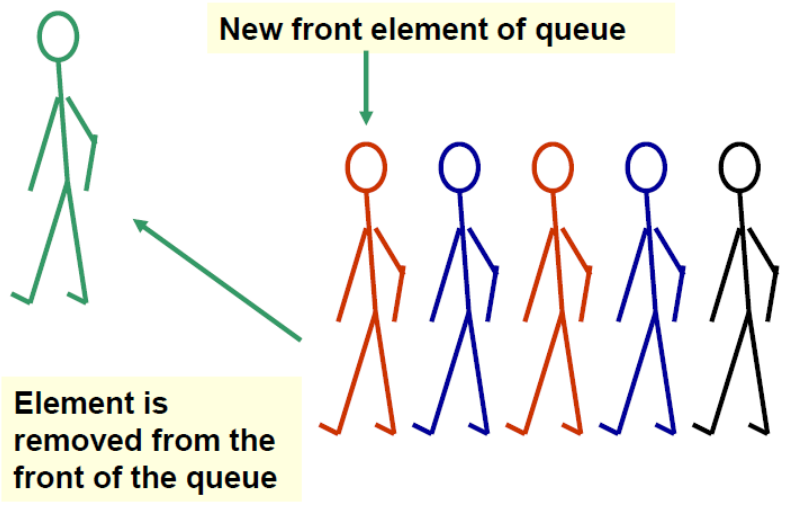


Conceptual view of a Queue

Adding an element



Removing an element



Uses of Queue in computing

- For any kind of problem involving FIFO Data.
- Printer Queue
- Keyboard Input buffer
- GUI Event Queue
- Web browser request Queue

Uses of Queues in computing

- In simulation studies, where the goal is to reduce waiting times:
  - Optimize the flow of traffic at a traffic light
  - Determine the number of cashiers to have on duty at a grocery store at different times of day
- Process Scheduling
- Printer Queues in Network

Queue Operations

- Enqueue(x) or push(x): add an element to the tail of a queue.
- Dequeue() or pop(): remove an element from the head of a queue.
- Front() or peek(): examine the element at the head of the queue.
- QEmpty() or IsEmpty()
- QFull() or IsFull()
- **It is *not legal to access the elements in the middle of the queue!***

Implementation of Queues

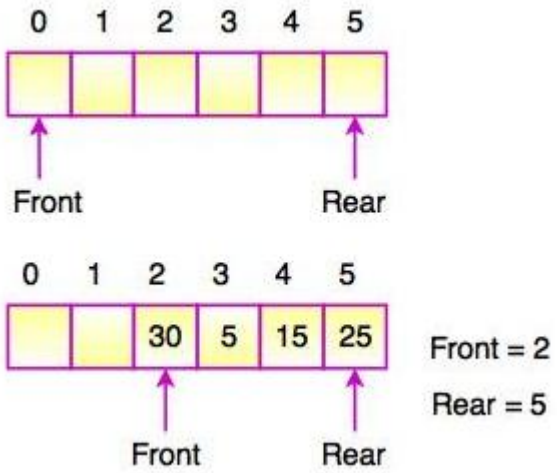
- Array
- Linked List

Array Implementation

```
int A[SIZE]
front <- -1
rear <- -1
```

```
Enqueue(x)
{
  if( IsFull() )
     printf("Queue is Full")
  else if ( IsEmpty() )
       {
         front <- read <- 0
       }
  else
       {
         rear <- rear + 1
       }
       A[rear] <- x
}
```

```
Dequeue()
{
  if( IsEmpty() )
     printf("Queue is Empty")
  else if (front == rear )
       {
         front <- read <- -1
       }
  else
       {
         front <- front + 1
       }
}
```




```
IsFull()
{
  if ( rear == size(a)-1 )
     return True
  else
     return False
}
```

```
IsEmpty()
{
  if ( front == -1 && rear == -1 )
     return True
  else
     return False
}
```

C Code

```
#include <stdio.h>
#define MAX 50
int queue_array[MAX];
int rear = - 1;
int front = - 1;

display()
{
   int i;
   if (front == - 1)
      printf("Queue is empty \n");
   else
   {
      printf("Queue is : \n");
      for (i = front; i <= rear; i++)
         printf("%d ", queue_array[i]);
      printf("\n");
   }
} /*End of display() */
```

```
main()
{
   int choice;
   while (1)
   {
      printf("1.Insert \n");
      printf("2.Delete\n");
      printf("3.Display \n");
      printf("4.Exit \n");
      printf("Enter your choice : ");
      scanf("%d", &choice);
      switch (choice)
      {
         case 1:
         insert();
         break;
         case 2:
         delete();
         break;
         case 3:
         display();
         break;
         case 4:
         exit(1);
         default:
         printf("Inavlid choice \n");
      } /*End of switch*/
   } /*End of while*/
} /*End of main()*/
```

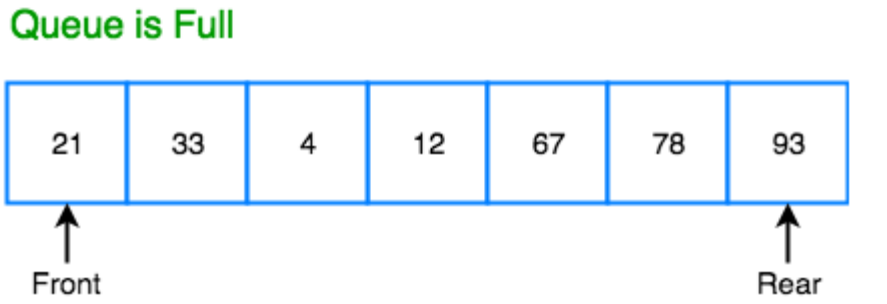
C code

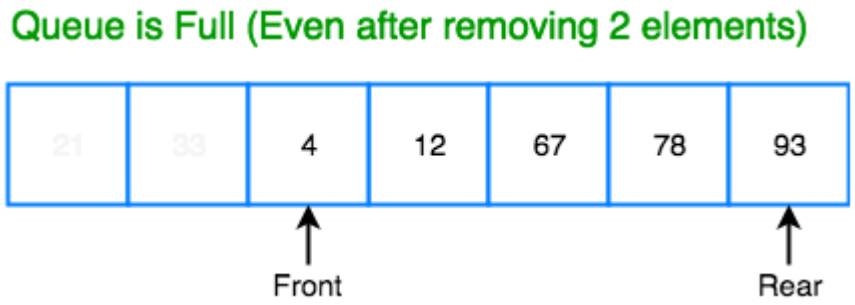
```
insert()
{
   int add_item;
   if (rear == MAX - 1)
   printf("Queue Overflow \n");
   else
   {
      if (front == - 1)
      /*If queue is initially empty */
      front = 0;
      printf("Inset the element in queue : ");
      scanf("%d", &add_item);
      rear = rear + 1;
      queue_array[rear] = add_item;
   }
} /*End of insert()*/
```

```
delete()
{
   if (front == - 1 || front > rear)
   {
      printf("Queue Underflow \n");
      return ;
   }
   else
   {
      printf("Deleted Element is : %d\n", queue_array[front]);
      front = front + 1;
   }
} /*End of delete() */
```

Circular Queue

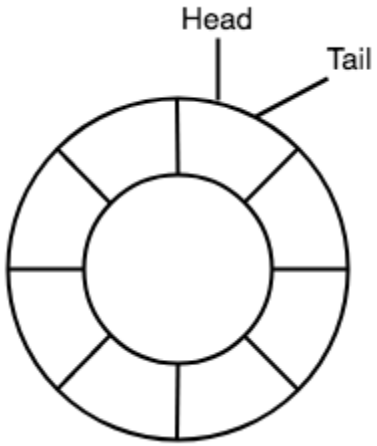
- Why we need a circular queue, when we already have linear queue data structure.





Circular Queue

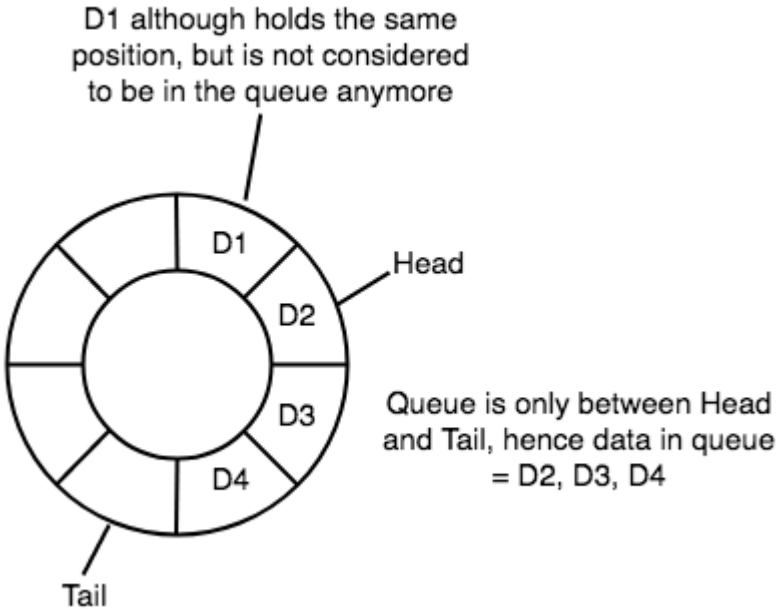
- **Circular Queue** is also a linear data structure, which follows the principle of **FIFO**(First In First Out), but instead of ending the queue at the last position, it again starts from the first position after the last, hence making the queue behave like a circular data structure.



A simple circular queue with size 8

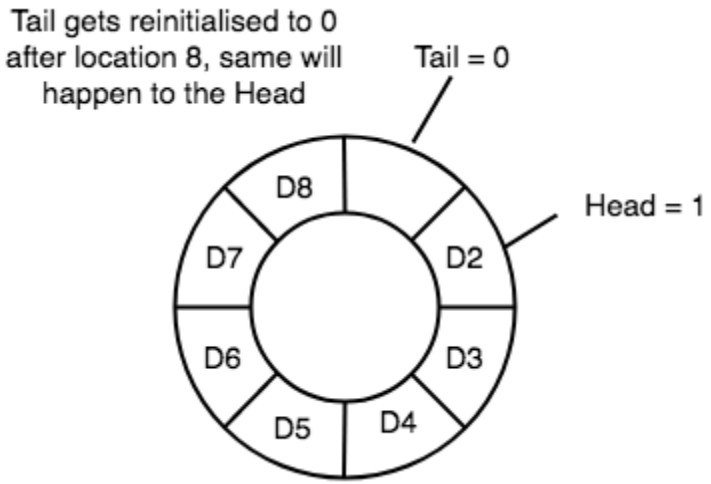
Circular Queue

- In a circular queue, data is not actually removed from the queue. Only the head pointer is incremented by one position when **dequeue** is executed. As the queue data is only the data between head and tail, hence the data left outside is not a part of the queue anymore, hence removed.



Circular Queue

- The head and the tail pointer will get reinitialised to **0** every time they reach the end of the queue.
- Current Position = i
- Next Position = (i+1)% N
- Previous Position = (i+N-1)%N



Application of Circular Queue

- Below we have some common real-world examples where circular queues are used:
- Computer controlled **Traffic Signal System** uses circular queue.
- CPU scheduling and Memory management.

Implementation of Circular Queue

Below we have the implementation of a circular queue:

1. Initialize the queue, with size of the queue defined (`maxSize`), and `head` and `tail` pointers.
2. `enqueue` : Check if the number of elements is equal to maxSize - 1:
    - If **Yes**, then return **Queue is full**.
    - If **No**, then add the new data element to the location of `tail` pointer and increment the `tail` pointer.
3. `dequeue` : Check if the number of elements in the queue is zero:
    - If **Yes**, then return **Queue is empty**.
    - If **No**, then increment the `head` pointer.
4. Finding the `size` :
    - If, **tail >= head**, `size = (tail - head) + 1`
    - But if, **head > tail**, then `size = maxSize - (head - tail) + 1`

Implementation of Circular Queue

```
IsEmpty()
{
  if ( front == -1 && rear == -1 )
     return True
  else
     return False
}
```

```
int A[SIZE]
front <- -1
rear <- -1
```

```
Enqueue(x)
{
  if( IsFull() )
     printf("Queue is Full")
  else if ( IsEmpty() )
       {
         front <- read <- 0
       }
  else
      {
        rear <- rear + 1
      }
      A[rear] <- x
}
```

Queue



```
Enqueue(x)
{
  if( IsFull() )
     printf("Queue is Full")
  else if ( IsEmpty() )
       {
         front <- read <- 0
       }
  else
      {
        rear <- (rear + 1) % N
      }
      A[rear] <- x
}
```

Circular Queue

```
Dequeue()
{
  if( IsEmpty() )
     printf("Queue is Empty")
  else if (front == rear )
       {
         front <- read <- -1
       }
  else
      {
        front <- front + 1
      }
}
```

Queue



```
Dequeue()
{
  if( IsEmpty() )
     printf("Queue is Empty")
  else if (front == rear )
       {
         front <- read <- -1
       }
  else
      {
        front <- (front + 1) % N
      }
}
```

Circular Queue

```
IsFull()
{
  if ( rear == size(a)-1 )
     return True
  else
     return False
}
```

Queue



```
IsFull()
 {
   if ( (rear+1)%N == front )
      return True
   else
      return False
 }
```

Circular Queue

C Code – Circular Queue

```c
#include <stdio.h>

#define SIZE 5

int items[SIZE];
int front = -1, rear = -1;

// Check if the queue is full
int isFull() {
  if ((front == rear + 1) || (front == 0 && rear == SIZE - 1)) return 1;
  return 0;
}

// Check if the queue is empty
int isEmpty() {
  if (front == -1) return 1;
  return 0;
}
```

```c
// Adding an element
void enQueue(int element) {
  if (isFull())
    printf("\n Queue is full!! \n");
  else {
    if (front == -1) front = 0;
    rear = (rear + 1) % SIZE;
    items[rear] = element;
    printf("\n Inserted -> %d", element);
  }
}
```

```c
// Removing an element
int deQueue() {
  int element;
  if (isEmpty()) {
    printf("\n Queue is empty !! \n");
    return (-1);
  } else {
    element = items[front];
    if (front == rear) {
      front = -1;
      rear = -1;
    }
    // Q has only one element, so we reset the
    // queue after dequeing it. ?
    else {
      front = (front + 1) % SIZE;
    }
    printf("\n Deleted element -> %d \n", element);
    return (element);
  }
}
```

```c
// Display the queue
void display() {
  int i;
  if (isEmpty())
    printf(" \n Empty Queue\n");
  else {
    printf("\n Front -> %d ", front);
    printf("\n Items -> ");
    for (i = front; i != rear; i = (i + 1) % SIZE) {
      printf("%d ", items[i]);
    }
    printf("%d ", items[i]);
    printf("\n Rear -> %d \n", rear);
  }
}
```

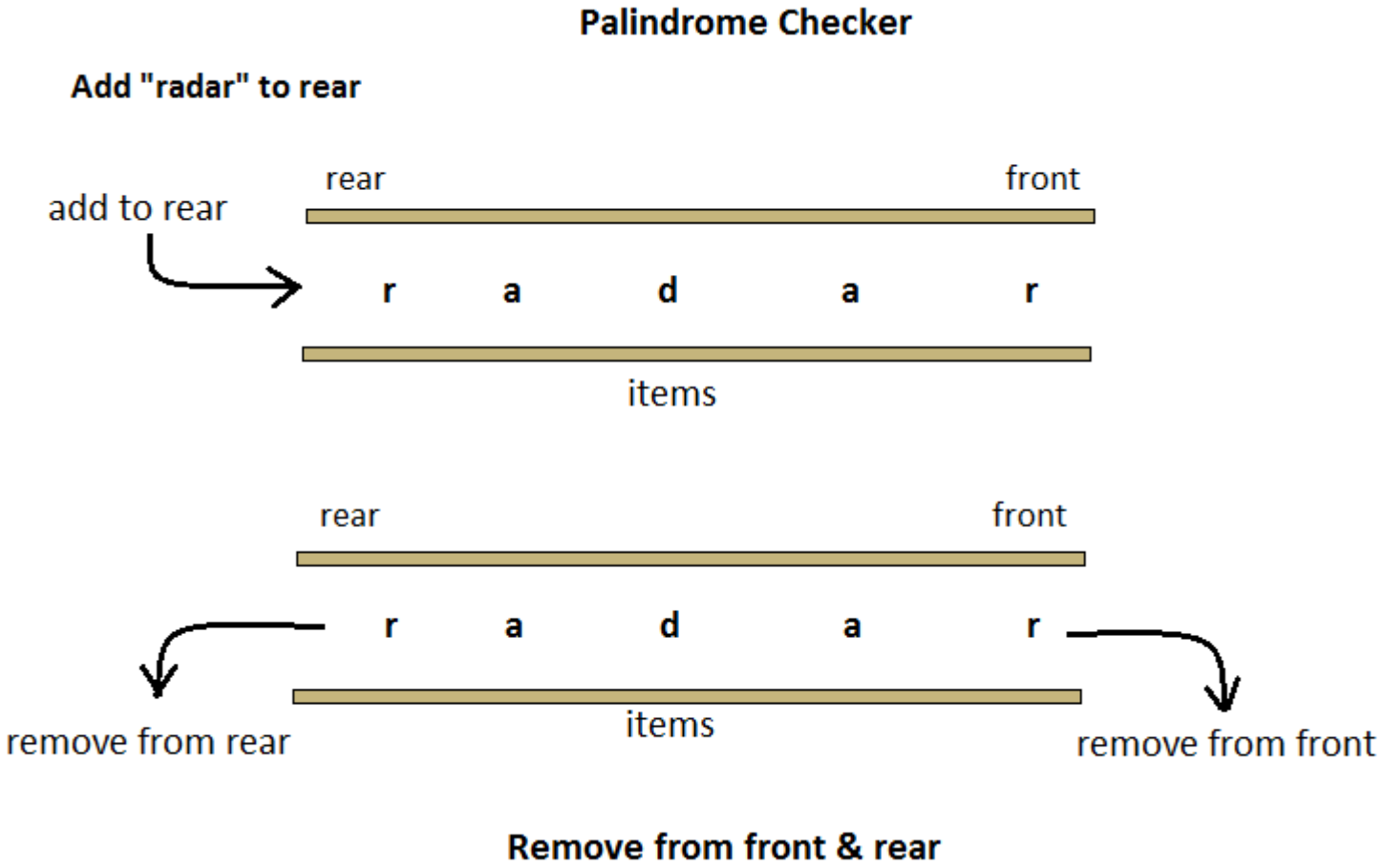
Double ended Queue

- Deque or Double Ended Queue is a type of queue in which insertion and removal of elements can be performed from either from the front or rear. Thus, it does not follow FIFO rule (First In First Out).



- Types of Deque
- Input Restricted Deque
  - In this deque, input is restricted at a single end but allows deletion at both the ends.
- Output Restricted Deque
  - In this deque, output is restricted at a single end but allows insertion at both the ends.

Applications



Algorithm

```
int A[SIZE]
front <- -1
rear <- -1
```

```
IsEmpty()
 {
   if ( front == -1 && rear == -1 )
      return True
   else
      return False
 }
```

```
IsFull()
 {
   if ( (rear+1)%N == front )
      return True
   else
      return False
 }
```

```
EnqueueR(x)
 {
   if( IsFull() )
      printf("Queue is Full")
   else if ( IsEmpty() )
        {
          front <- read <- 0
        }
   else
       {
         rear <- (rear + 1) % N
       }
       A[rear] <- x
 }
```

```
EnqueueF(x)
 {
   if( IsFull() )
      printf("Queue is Full")
   else if ( IsEmpty() )
        {
          front <- read <- 0
        }
   else
       {
         front <- (front + N - 1) % N
       }
       A[front] <- x
 }
```

```
DequeueF()
{
  if( IsEmpty() )
     printf("Queue is Empty")
  else if (front == rear )
       {
         front <- read <- -1
       }
  else
      {
        front <- (front + 1) % N
      }
}
```

```
DequeueR()
{
  if( IsEmpty() )
     printf("Queue is Empty")
  else if (front == rear )
       {
         front <- read <- -1
       }
  else
      {
        rear <- (rear + N - 1) % N
      }
}
```

C Code

```
/* DOUBLE ENDED QUEUE */

#include<stdio.h>
#include<conio.h>
#define MAX 10

int deque[MAX];
int left=-1, right=-1;

void insert_right(void);
void insert_left(void);
void delete_right(void);
void delete_left(void);
void display(void);
```

```
int main()
{
 int choice;
 clrscr();
 do
 {
  printf("\n1.Insert at right ");
  printf("\n2.Insert at left ");
  printf("\n3.Delete from right ");
  printf("\n4.Delete from left ");
  printf("\n5.Display ");
  printf("\n6.Exit");
  printf("\n\nEnter your choice ");
  scanf("%d",&choice);
  switch(choice)
  {
   case 1:
    insert_right();
    break;
   case 2:
    insert_left();
    break;
   case 3:
    delete_right();
    break;
   case 4:
    delete_left();
    break;
   case 5:
    display();
    break;
  }
 }while(choice!=6);
 getch();
 return 0;
}
```

```
//-------INSERT AT RIGHT-------
void insert_right()
{
 int val;
 printf("\nEnter the value to be added ");
 scanf("%d",&val);
 if( (left==0 && right==MAX-1) || (left==right+1) )
 {
  printf("\nOVERFLOW");
 }
 if(left==-1)        //if queue is initially empty
 {
  left=0;
  right=0;
 }
 else
 {
  if(right==MAX-1)
   right=0;
  else
   right=right+1;
 }
 deque[right]=val;
}
```

```
//-------INSERT AT LEFT-------
void insert_left()
{
 int val;
 printf("\nEnter the value to be added ");
 scanf("%d",&val);
 if( (left==0 && right==MAX-1) || (left==right+1) )
 {
  printf("\nOVERFLOW");
 }
 if(left==-1)        //if queue is initially empty
 {
  left=0;
  right=0;
 }
 else
 {
  if(left==0)
   left=MAX-1;
  else
   left=left-1;
 }
 deque[left]=val;
}
```

```
//-------DELETE FROM RIGHT-------
void delete_right()
{
 if(left==-1)
 {
  printf("\nUNDERFLOW");
  return;
 }
 printf("\nThe deleted element is %d\n", deque[right]);
 if(left==right)          //Queue has only one element
 {
  left=-1;
  right=-1;
 }
 else
 {
  if(right==0)
   right=MAX-1;
  else
   right=right-1;
 }
}
```

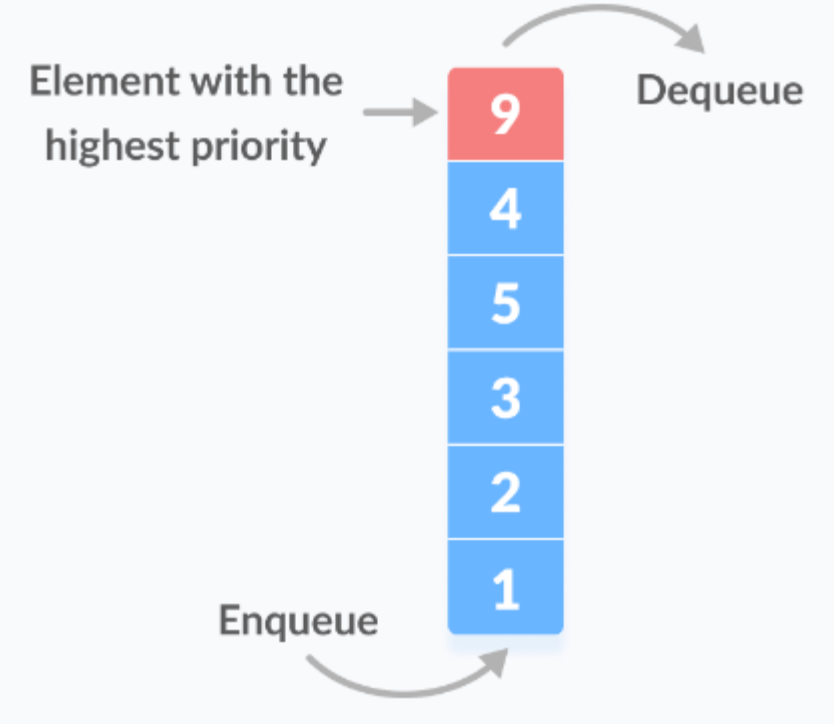
```
//-------DELETE FROM LEFT-------
void delete_left()
{
 if(left==-1)
 {
  printf("\nUNDERFLOW");
  return;
 }
 printf("\nThe deleted element is %d\n", deque[left]);
 if(left==right)          //Queue has only one element
 {
  left=-1;
  right=-1;
 }
 else
 {
  if(left==MAX-1)
   left=0;
  else
   left=left+1;
 }
}
```

Priority Queue

- Priority Queue is an extension of queue with following properties.
  - Every item has a priority associated with it.
  - An element with high priority is dequeued before an element with low priority.
  - If two elements have the same priority, they are served according to their order in the queue.

Priority Queue

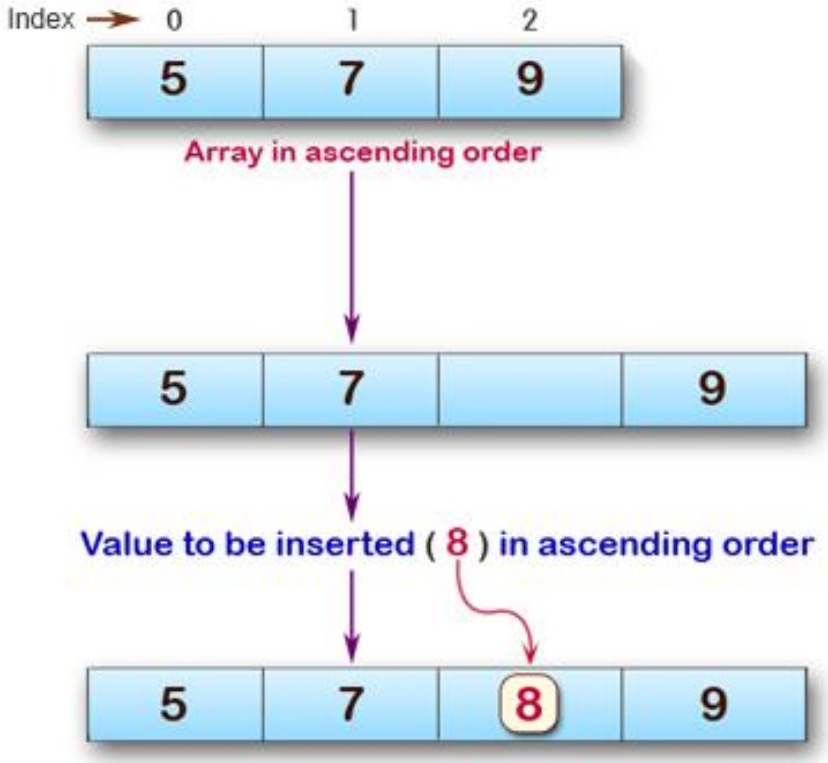
- In a queue, the **first-in-first-out rule** is implemented whereas, in a priority queue, the values are removed **on the basis of priority**. The element with the highest priority is removed first.



Types of Priority Queue

- **Min Priority Queue**: In min priority Queue minimum number of value gets the highest priority and lowest number of element gets the highest priority.
- **Max Priority Queue**: Max priority Queue is the opposite of min priority Queue in it maximum number value gets the highest priority and minimum number of value gets the minimum priority.

Solution


Index
0
1
2
5
7
9
Array in ascending order
5
7
9
Value to be inserted (8) in ascending order
5
7
8
9

```c
#include <stdio.h>

void main()
{
   int arr1[100],i,n,p,inval;
       printf("\n\nInsert New value in the sorted array :\n");
       printf("-----------------------------------------\n");

    printf("Input the size of array : ");
    scanf("%d", &n);
/* Stored values into the array*/
       printf("Input %d elements in the array in ascending order:\n",n);
       for(i=0;i<n;i++)
            {
          printf("element - %d : ",i);
          scanf("%d",&arr1[i]);
        }
   printf("Input the value to be inserted : ");
   scanf("%d",&inval);
   printf("The exist array list is :\n ");
   for(i=0;i<n;i++)
      printf("% 5d",arr1[i]);
```

```
for(i=0;i<n;i++)
   printf("% 5d",arr1[i]);
/* Determine the position where the new value will be insert.*/
for(i=0;i<n;i++)
  if(inval<arr1[i])
  {
    p = i;
    break;
  }
/* move all data at right side of the array */
for(i=n;i>=p;i--)
   arr1[i]= arr1[i-1];
/* insert value at the proper position */
   arr1[p]=inval;

   printf("\n\nAfter Insert the list is :\n ");
for(i=0;i<=n;i++)
   printf("% 5d",arr1[i]);
   printf("\n");
```

C Code

```
/*
 * C Program to Implement Priority Queue to Add and Delete Elements
 */
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

void insert_by_priority(int);
void delete_by_priority(int);
void create();
void check(int);
void display_pqueue();

int pri_que[MAX];
int front, rear;

void main()
{
    int n, ch;

    printf("\n1 - Insert an element into queue");
    printf("\n2 - Delete an element from queue");
    printf("\n3 - Display queue elements");
    printf("\n4 - Exit");

    create();

    while (1)
    {
        printf("\nEnter your choice : ");
        scanf("%d", &ch);

        switch (ch)
        {
        case 1:
            printf("\nEnter value to be inserted : ");
            scanf("%d",&n);
            insert_by_priority(n);
            break;
        case 2:
            printf("\nEnter value to delete : ");
            scanf("%d",&n);
            delete_by_priority(n);
            break;
        case 3:
            display_pqueue();
            break;
        case 4:
            exit(0);
        default:
            printf("\nChoice is incorrect, Enter a correct choice");
        }
    }
}
```

C Code

```
/* Function to create an empty priority queue */
void create()
{
    front = rear = -1;
}

/* Function to insert value into priority queue */
void insert_by_priority(int data)
{
    if (rear >= MAX - 1)
    {
        printf("\nQueue overflow no more elements can be inserted");
        return;
    }
    if ((front == -1) && (rear == -1))
    {
        front++;
        rear++;
        pri_que[rear] = data;
        return;
    }
    else
        check(data);
    rear++;
}
```

```
/* Function to check priority and place element */
void check(int data)
{
    int i,j;

    for (i = 0; i <= rear; i++)
    {
        if (data >= pri_que[i])
        {
            for (j = rear + 1; j > i; j--)
            {
                pri_que[j] = pri_que[j - 1];
            }
            pri_que[i] = data;
            return;
        }
    }
    pri_que[i] = data;
}
```

C Code

```
/* Function to delete an element from queue */
void delete_by_priority(int data)
{
    int i;

    if ((front==-1) && (rear==-1))
    {
        printf("\nQueue is empty no elements to delete");
        return;
    }

    for (i = 0; i <= rear; i++)
    {
        if (data == pri_que[i])
        {
            for (; i < rear; i++)
            {
                pri_que[i] = pri_que[i + 1];
            }

        pri_que[i] = -99;
        rear--;

        if (rear == -1)
            front = -1;
        return;
        }
    }
    printf("\n%d not found in queue to delete", data);
}
```

```
/* Function to display queue elements */
void display_pqueue()
{
    if ((front == -1) && (rear == -1))
    {
        printf("\nQueue is empty");
        return;
    }

    for (; front <= rear; front++)
    {
        printf(" %d ", pri_que[front]);
    }

    front = 0;
}
```

Priority Queue (Circular Q concept)

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 30

typedef struct pqueue
{
    int data[MAX];
    int rear,front;
}pqueue;

void initialize(pqueue *p);
int empty(pqueue *p);
int full(pqueue *p);
void enqueue(pqueue *p, int x);
int dequeue(pqueue *p);
void print(pqueue *p);

void initialize(pqueue *p)
{
    p->rear=-1;
    p->front=-1;
}

int empty(pqueue *p)
{
    if(p->rear==-1)
        return(1);

    return(0);
}

int full(pqueue *p)
{
    if((p->rear+1)%MAX==p->front)
        return(1);

    return(0);
}
```

```c
int dequeue(pqueue *p);
void print(pqueue *p);

void main()
{
    int x,op,n,i;
    pqueue q;
    initialize(&q);

    do
    {
        printf("\n1)Create \n2)Insert \n3)Delete \n4)Print \n5)EXIT");
        printf("\nEnter Choice: ");
        scanf("%d",&op);
        switch (op) {
            case 1: printf("\nEnter Number of Elements");
                    scanf("%d",&n );
                    initialize(&q);
                    printf("Enter the data");

                    for(i=0; i<n; i++)
                    {
                        scanf("%d",&x);
                        if(full(&q))
                        {
                            printf("\nQueue is Full..");
                            exit(0);
                        }
                        enqueue(&q,x);
                    }
                    break;

            case 2: printf("\nEnter the element to be inserted");
                    scanf("%d\n",&x);
                    if(full(&q))
                    {
                        printf("\nQueue is Full");
                        exit(0);
                    }
                    enqueue(&q,x);
                    break;

            case 3: if(empty(&q))
                    {
                        printf("\nQueue is empty..");
                        exit(0);
                    }

                    x=dequeue(&q);
                    printf("\nDeleted Element=%d",x);
                    break;

            case 4: print(&q);
                    break;
            default: break;

            }
        }while (op!=5);
    }
```

```
void enqueue(pqueue *p, int x)
{
    int i;
    if(full(p))
        printf("\nOverflow");
    else
    {
        if(empty(p))
        {
            p->rear=p->front=0;
            p->data[0]=x;
        }
        else
        {
            i=p->rear;

            while(x>p->data[i])
            {
                p->data[(i+1)%MAX]=p->data[i];
                i=(i-1+MAX)%MAX; //anticlockwise movement inside the queue
                if((i+1)%MAX==p->front)
                    break;
            }

            //insert x
            i=(i+1)%MAX;
            p->data[i]=x;

            //re-adjust rear
            p->rear=(p->rear+1)%MAX;
        }
    }
}
```

```
int dequeue(pqueue *p)
{
    int x;

    if(empty(p))
    {
        printf("\nUnderflow..");
    }
    else
    {
        x=p->data[p->front];
        if(p->rear==p->front)   //delete the last element
            initialize(p);
        else
            p->front=(p->front +1)%MAX;
    }

    return(x);
}
```

```
void print(pqueue *p)
{
    int i,x;

    if(empty(p))
    {
        printf("\nQueue is empty..");
    }
    else
    {
        i=p->front;

        while(i!=p->rear)
        {
            x=p->data[i];
            printf("\n%d",x);
            i=(i+1)%MAX;
        }

        //prints the last element
        x=p->data[i];
        printf("\n%d",x);
    }
}
```